

Thread in C

Tipo pthread

```
pthread_t Nome_thread;
```

"pthread" è un tipo (come lo è int o char) solo che non è un tipo primitivo di C ma è definito nella libreria `#include <pthread.h>`

Funzione pthread_create

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *  
(*start_routine) (void *), void *arg);
```

Crea un nuovo thread di esecuzione all'interno del processo chiamante. Il nuovo thread inizia a girare in modo concorrente (parallelo) rispetto al thread principale, partendo dall'inizio della funzione passata come terzo parametro (`start_routine`).

Vediamo i parametri:

- `pthread_t *thread`: È un puntatore alla variabile che conterrà l'identificatore del thread. La funzione scrive in questa cella di memoria l'ID del thread appena creato, permettendoti di usarlo in seguito (ad esempio nella `pthread_join`).
 - `const pthread_attr_t *attr`: È un puntatore a una struttura che definisce i requisiti del thread (es. la dimensione dello stack o la priorità). Passando `NULL`, dici al sistema operativo di ignorare configurazioni personalizzate e usare le impostazioni standard.
 - `void *(*start_routine) (void *)`: È il puntatore alla funzione che il thread deve eseguire. In pratica, passi il nome della funzione che diventerà il "programma principale" di quel thread.
 - `void *arg`: È il parametro di input da passare alla funzione eseguita dal thread. È un puntatore generico (`void *`), il che significa che puoi passarci l'indirizzo di qualsiasi cosa (un intero, una stringa, una struttura dati complessa). Passando `NULL`, indichi che la funzione non ha bisogno di argomenti.
-

Funzione pthread_detach

Normalmente un thread è in modalità *joinable* il che significa che una volta finito il suo compito, la memoria allocata per lui resta ancora riservata anche se il thread ha finito il suo lavoro. Con `thread_detach(nomeThread)` rendiamo il thread *detached* in questa maniera una volta finito il suo lavoro libererà automaticamente le risorse a lui allocate.

Funzione pthread_join

```
int pthread_join(pthread_t thread, void **retval);
```

Sospende l'esecuzione del thread chiamante (di solito il `main`) finché il thread specificato dal parametro `thread` non termina la sua esecuzione. È l'equivalente della funzione `wait()` che si usa per i processi.

Vediamo i parametri:

- `pthread_t thread`: È l'ID del thread che vuoi aspettare. Qui si passa il valore diretto (non il puntatore con `&`), indicando esattamente al sistema quale "filo" deve terminare prima di far proseguire il programma principale.
- `void retval`: È un puntatore a un puntatore. Serve a ricevere il valore di ritorno che la funzione del thread restituisce quando termina (con l'istruzione `return`). Passando `NULL`, dici che non ti interessa intercettare il valore di chiusura del thread, ma vuoi solo aspettare che finisca.

Funzione pthread_mutex_init

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

La funzione `pthread_mutex_init` serve a **inizializzare un Mutex**

I parametri sono:

- `pthread_mutex_t *mutex`: È il puntatore alla variabile di tipo `pthread_mutex_t` che vuoi inizializzare (nel tuo caso, l'indirizzo della memoria `&clients.mutex`). La funzione imposta lo stato iniziale di questo lucchetto come "aperto/sbloccato".
- `const pthread_mutexattr_t *attr`: È un puntatore a una struttura che definisce gli attributi e il comportamento del lucchetto (ad esempio, se il lucchetto può essere ricorsivo). Passando `NULL`, stai dicendo al sistema di usare gli **attributi predefiniti** (un lucchetto normale standard).

In alternativa posso usare la macro: `PTHREAD_MUTEX_INITIALIZER` (anche fuori dal `main`) come in questo esempio: `pthread_mutex_t mutexLog = PTHREAD_MUTEX_INITIALIZER;`

A livello pratico...

```
pthread_mutex_lock(&clients.mutex); // 1. Chiudo il lucchetto. Se è
già chiuso da un altro thread, mi fermo e aspetto.

// --- ZONA SICURA (Critical Section) ---
// Qui modifico o leggo la struttura 'clients' in totale sicurezza.
// Nessun altro thread può toccarla finché sono qui dentro.
// -----

pthread_mutex_unlock(&clients.mutex); // 2. Riapro il lucchetto,
permettendo ad altri thread di usarlo.
```

La sezione critica si gestisce trami 2 funzioni:

`pthread_mutex_lock(&sensori.mutex);` e
`pthread_mutex_unlock(&sensori.mutex);` nel mezzo il codice della sezione critica.

Funzioni eseguite dai thread

`void * nomeFunzione(void *argomenti)`

la libreria `pthread.h` impone una regola rigida: qualsiasi funzione tu voglia dare in pasto a un thread deve avere tassativamente questa identica struttura